



C#.NET

Mr. Bangar Raju

 **NARESHO**
technologies

Inheritance (Part-II)

Understanding Inheritance in C#: Key Concepts and Rules

By Albert Harmon

Published August 24, 2016

🕒 3 min read

Education



TLDR This blog post explores the fundamental principles of inheritance in C#, detailing six essential rules that govern how classes interact within the inheritance hierarchy, including constructor

accessibility, member access, and types of inheritance.

Inheritance is a fundamental concept in object-oriented programming, allowing classes to inherit properties and methods from other classes. In this blog post, we will delve into the key principles of inheritance in C#, highlighting six essential rules that every developer should understand.

Recap of Previous Points

In the previous session, we discussed three important points regarding inheritance:

1. **Parent Class Constructor Accessibility:** The constructor of the parent class must be accessible to the child class. This is crucial because when an instance of the child class is created, the child class constructor implicitly calls the parent class constructor to initialize parent class variables.
2. **Member Access:** Child classes can access members of the parent class, but the reverse is not true; parent classes cannot access members of their child classes. This reflects the principle that children can utilize their parents' properties, but parents do not have rights over their children's properties.
3. **Parent Class Variable Initialization:** A parent class variable can be initialized using a child class instance. However, while doing so, the

child class members cannot be accessed through the parent class reference.

Fourth Rule: Default Parent Class

Every class defined in C# has a default parent class, which is the `Object` class located in the `System` namespace. This means that even if you do not explicitly inherit from any class, your class will inherit from `Object` by default. The `Object` class provides four important methods that are accessible from any class:

- `Equals`
- `GetHashCode`
- `GetType`
- `ToString`

These methods are inherited by all classes, ensuring that every class has these fundamental functionalities.

Fifth Rule: Types of Inheritance

Inheritance can be categorized based on the number of parent and child classes involved. According to C++, which laid the groundwork for object-oriented programming, there are five types of inheritance:

1. **Single Inheritance:** One child class inherits from one parent class.

2. **Multi-level Inheritance:** A class inherits from another class, forming a chain of inheritance.
3. **Hierarchical Inheritance:** One parent class has multiple child classes.
4. **Hybrid Inheritance:** A combination of multi-level and hierarchical inheritance.
5. **Multiple Inheritance:** A child class inherits from multiple parent classes.

However, in C#, multiple inheritance through classes is not supported. Instead, C# allows single inheritance, multi-level inheritance, and hierarchical inheritance, but not hybrid or multiple inheritance.

Sixth Rule: Constructor Calling in Inheritance

When a child class instance is created, the child class constructor implicitly calls its parent class constructor. If the parent class constructor is parameterless, this process occurs automatically. However, if the parent class constructor requires parameters, the child class must explicitly call it using the `base` keyword. This ensures that the necessary values are passed to the parent class constructor.

Example of Explicit Constructor Calling

If the parent class has a parameterized constructor, the child class must provide the required arguments:

```
class Parent
{
    public Parent(int value) {
        // Initialization code
    }
}

class Child : Parent
{
    public Child(int value) : base(value) {
        // Child constructor code
    }
}
```

In this example, the `Child` class constructor explicitly calls the `Parent` class constructor and passes the required value.

Conclusion

Understanding these six rules of inheritance is crucial for effective programming in C#. By grasping how constructors work, the significance of the `object` class, and the types of inheritance, developers can create more robust and maintainable code. Inheritance not only promotes code reusability but also establishes a clear hierarchy among classes, making it easier to manage complex systems. Thank you for joining this session on inheritance in C#!